

Algorithm Design and Analysis - Quiz 3 Rubric

Supratim Shit, Syamantak Das, Divyajeet Singh

1. (20 points) Define the *inner product* of two matrices $X, Y \in \mathbb{R}^{p \times q}$ as a function $\langle \cdot, \cdot \rangle$ that returns a scalar value

$$\langle X, Y \rangle = \sum_{i=1}^p \sum_{j=1}^q X[i, j] \cdot Y[i, j] \quad (1)$$

Define a *row-matrix* of a matrix $M \in \mathbb{R}^{p \times q}$ as a matrix $S \in \mathbb{R}^{r \times q}$ formed by using r consecutive rows of M .

You are given matrices $A \in \mathbb{R}^{n \times d}$ and $B \in \mathbb{R}^{m \times d}$. Your task is to find the maximum inner product that can be obtained between row-matrices of A and B of the same size.

Hint: Think of the case when $d = 1$, i.e. when A and B are arrays of size n and m respectively. How would you find the maximum inner product between continuous subsequences of A and B ?

Note: I meant 'sub-arrays' instead of subsequences, but the word continuous implies the same restriction.

Intuition:

Following the hint, think of the case when A and B are arrays of length n and m respectively. You want to find the maximum *dot product* between the *vectors* (in $\mathbb{R}^r$, for some $1 \leq r \leq \min(n, m)$) formed by continuous sub-arrays of A and B . Note that by the given definition of row-matrix and equation (1), we have for $d = 1$ and $X, Y \in \mathbb{R}^{r \times 1}$

$$\langle X, Y \rangle = \sum_{i=1}^r X[i, 1] \cdot Y[i, 1] \quad (\text{since } d \text{ can only be } 1) \quad (2)$$

which is the standard vector dot product. Then, this is a standard dynamic programming problem. We can use a 2-dimensional DP where $dp[i, j]$ represents the maximum dot product possible between sub-arrays of A ending at $A[i]$ and of B ending at $B[j]$.

Extending this to the multi-dimensional setting, we can use the same approach! $dp[i, j]$ now represents the maximum inner product possible when considering row-matrices that end at $A[i]$ and $B[j]$. Note that we need not necessarily use *all* i or j rows of A or B respectively. We only need to select some $1 \leq r \leq \min(i, j)$ continuous rows from A and B (which also need not necessarily start from index 1). This r need not be explicitly found. Once you have an inner product of some r rows of A and B , you can *extend* it by one more row by adding the dot product between the next rows of A and B .

- (a) (2 points) Give a precise description of the sub-problems you want to solve in the dynamic program.

Note: Here, we require 'what is the sub-problem,' and not how it was obtained or how it will be solved.

Define $T[i, j]$ as the maximum inner product between row-matrices of the same size from $A[1 : i]$ that end at $A[i]$ and from $B[1 : j]$ that end at $B[j]$, where $1 \leq i \leq n$ and $1 \leq j \leq m$.

Alternatively, we can start the dynamic program from the bottom. Define $T[i, j]$ as the maximum inner product between the row-matrices of the same size from $A[n - i + 1 : n]$ that end at $A[n - i + 1]$ and from $B[m - j + 1 : m]$ that end at $B[m - j + 1]$, where $1 \leq i \leq n$ and $1 \leq j \leq m$.

Alternatively, we can use a more complicated DP state. Define $T[i, j, r]$ as the inner product between the row-matrices of exactly r rows, i.e. $A[r - i : i]$ and $B[r - j : j]$, where $1 \leq i \leq n$, $1 \leq j \leq m$, and $\min(i, j) \leq r \leq \min(n, m)$.

Grading:

- (Full marks) 2 points for a correct definition. Mentioning ‘of the same size’ (for row-matrices) is not necessary.
Note to TAs: Be careful. It is possible to define $T[i, j]$ where i indexes B and j indexes A , which is also correct. The reverse of the orange solution is also possible.
- (Partial marks) Deduct 0.5 points for every missing condition (on i , j , and/or r).
- (No marks) 0 points otherwise.

- (b) (2 points) Give a recurrence relation (with base case) which relates each sub-problem to “smaller” sub-problems with a one-line description of what it represents.

Note: Here, we just require the recurrence, not the algorithm or why the recurrence is correct.

Using the first definition of $T[i, j]$, the recurrence relation is given by

$$T[i, j] = \max \begin{cases} \langle A[i], B[j] \rangle & \text{Start a new inner product from } A[i] \text{ and } B[j] \\ T[i - 1, j - 1] + \langle A[i], B[j] \rangle & \text{Include } \langle A[i], B[j] \rangle \text{ in the existing inner product} \end{cases} \quad (3)$$

The base case of the recurrence is

$$T[i, 0] = T[0, j] = -\infty \quad \forall 1 \leq i \leq n, 1 \leq j \leq m \quad (4)$$

Alternatively, using the *reversed* definition of $T[i, j]$,

$$T[i, j] = \max \begin{cases} \langle A[i], B[j] \rangle & \text{Start a new inner product from } A[i] \text{ and } B[j] \\ T[i + 1, j + 1] + \langle A[i], B[j] \rangle & \text{Include } \langle A[i], B[j] \rangle \text{ in the existing inner product} \end{cases} \quad (5)$$

The base case of this recurrence is

$$T[i, n + 1] = T[m + 1, j] = -\infty \quad \forall 1 \leq i \leq n, 1 \leq j \leq m \quad (6)$$

Alternatively, using the definition of $T[i, j, r]$,

$$T[i, j, r] = T[i - 1, j - 1, r - 1] + \langle A[i], B[j] \rangle \quad \text{Extend the previous } (r - 1)\text{-sized inner product} \quad (7)$$

The base case of this recurrence is

$$T[i, 0, 0] = T[0, j, 0] = 0 \quad \forall 1 \leq i \leq n, 1 \leq j \leq m \quad (8)$$

Since $A[i]$ and $B[j]$ are vectors in $\mathbb{R}^d$, we use the notation $\langle A[i], B[j] \rangle$ to denote the dot product between them,

$$\langle A[i], B[j] \rangle = \sum_{k=1}^d A[i, k] \cdot B[j, k] \quad (9)$$

Note to TAs: Be careful with the indexing. My solutions use 1-based indexing for A and B . The blue and orange solutions use 0-based indexing for T . The orange solution uses 0-based indexing for r . The green solution uses 1-based indexing for T .

Grading:

- 1 point for a correct recurrence relation. 0.5 points if one of the recurrence case is missing. 0 points otherwise.
- 1 point for the correct base cases (both row & column). 0.5 points if either the row or the column case is missing. 0 points otherwise.

(c) (1 point) Identify the sub-problem that solves the original problem.

Based on the definition of sub-problems/dynamic programming states, the sub-problems that solve the original problem (the final answer) is given by

$$\max_{1 \leq i \leq n, 1 \leq j \leq m} T[i, j] \quad (10)$$

which is the maximum inner product possible using row-matrices of the same size that could be formed using all n rows of A and all m rows of B .

The above also holds for the green solution. The maximum inner product between row-matrices of the same size using all n rows of A and all m rows of B is given by (10).

Alternatively, using the 3-dimensional state definition, the maximum inner product possible using row-matrices of any size r that could be formed using all n rows of A and all m rows of B will be given by

$$\max_{1 \leq i \leq n, 1 \leq j \leq m, \min(i, j) \leq r \leq \min(n, m)} T[i, j, r] \quad (11)$$

Grading:

- (Full marks) 1 point for specifying the correct maximum expression.
- (No marks) 0 points otherwise.
- No partial marking.

(d) (10 points) Give the pseudocode for implementing your dynamic program **iteratively**. A C++/Java-like program with heavy syntax, variable declarations, etc. **will not be accepted**.

Note: You do not need to prove the correctness of your algorithm in any form.

Algorithm 1 shows the implementation of the blue solution. Since we require (at least) $T[i - 1, j - 1]$ to compute $T[i, j]$, we fill the table from the top left to the bottom right.

Alternatively, Algorithm 2 shows the implementation of the green solution. Since we require (at least) $T[i + 1, j + 1]$ to compute $T[i, j]$, we fill the table from the bottom right to the top left.

Alternatively, Algorithm 3 shows the implementation of the orange solution. Since we require (at least) $T[i - 1, j - 1, r - 1]$ to compute $T[i, j, r]$, we fill the table in a forward manner, i.e. we fill lower (i, j, r) triplets first.

Note: You may choose to pre-compute the row-wise dot products and store them in a table $C[i, j] = \langle A[i], B[j] \rangle$ for $1 \leq i \leq n$ and $1 \leq j \leq m$ (which is done in $O(nmd)$ time), and use them instead of calculating each $\langle A[i], B[j] \rangle$ on the fly. This does not change the overall runtime.

Algorithm 1 Iterative (forward) dynamic program to find the maximum inner product between row-matrices of $A \in \mathbb{R}^{n \times d}$ and $B \in \mathbb{R}^{m \times d}$

```

1: procedure MAXIMUM-INNER-PRODUCT( $A[1 : n], B[1 : m]$ )  $\triangleright A[i], B[j] \in \mathbb{R}^d$ 
2:    $T[0 : n][0 : m] \leftarrow -\infty$   $\triangleright (n + 1) \times (m + 1)$  DP table
3:   for  $i \leftarrow 1$  to  $n$  do
4:     for  $j \leftarrow 1$  to  $m$  do
5:        $c \leftarrow 0$ 
6:       for  $k \leftarrow 1$  to  $d$  do
7:          $c \leftarrow c + A[i, k] \cdot B[j, k]$ 
8:       end for
9:        $T[i, j] \leftarrow \max(c, T[i - 1, j - 1] + c)$ 
10:    end for
11:  end for
12:   $c^* \leftarrow -\infty$ 
13:  for  $i \leftarrow 1$  to  $n$  do
14:    for  $j \leftarrow 1$  to  $m$  do
15:       $c^* \leftarrow \max(c^*, T[i, j])$ 
16:    end for
17:  end for
18:  return  $c^*$ 
19: end procedure

```

Algorithm 2 Iterative (backward) dynamic program to find the maximum inner product between row-matrices of $A \in \mathbb{R}^{n \times d}$ and $B \in \mathbb{R}^{m \times d}$

```

1: procedure MAXIMUM-INNER-PRODUCT( $A[1 : n], B[1 : m]$ )  $\triangleright A[i], B[j] \in \mathbb{R}^d$ 
2:    $T[1 : n + 1][1 : m + 1] \leftarrow -\infty$   $\triangleright (n + 1) \times (m + 1)$  DP table
3:   for  $i \leftarrow n$  to  $1$  do
4:     for  $j \leftarrow m$  to  $1$  do
5:        $c \leftarrow \sum_{k=1}^d A[i, k] \cdot B[j, k]$ 
6:        $T[i, j] \leftarrow \max(c, T[i + 1, j + 1] + c)$ 
7:     end for
8:   end for
9:   return  $\max_{1 \leq i \leq n, 1 \leq j \leq m} T[i, j]$ 
10: end procedure

```

Algorithm 3 Iterative (3D) dynamic program to find the maximum inner product between row-matrices of $A \in \mathbb{R}^{n \times d}$ and $B \in \mathbb{R}^{m \times d}$

```

1: procedure MAXIMUM-INNER-PRODUCT( $A[1 : n], B[1 : m]$ )  $\triangleright A[i], B[j] \in \mathbb{R}^d$ 
2:    $r_{\max} \leftarrow \min(n, m)$ 
3:    $T[0 : n][0 : m][0 : r_{\max}] \leftarrow -\infty$   $\triangleright (n + 1) \times (m + 1) \times (\min(n, m) + 1)$  DP table
4:    $T[0, j, 0] = T[i, 0, 0] \leftarrow 0 \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$ 
5:    $c^* \leftarrow -\infty$ 
6:   for  $r \leftarrow 1$  to  $r_{\max}$  do
7:     for  $i \leftarrow 1$  to  $n$  do
8:       for  $j \leftarrow 1$  to  $m$  do
9:          $T[i, j, r] \leftarrow T[i - 1, j - 1, r - 1] + \langle A[i], B[j] \rangle$ 
10:         $c^* \leftarrow \max(c^*, T[i, j, r])$ 
11:      end for
12:    end for
13:  end for
14:  return  $c^*$ 
15: end procedure

```

Grading:

- (Full marks) 10 points for any fully correct **iterative** DP pseudocode.
- (Partial marks for blue and green solutions) 2 points for initializing the DP table with the correct base case. 1 point **for each** for loop (on n and m). 2 points for calculating the current $\langle A[i], B[j] \rangle$. 2 points for filling the DP table with the correct recurrence relation. 2 points for calculating/returning the correct final answer.

Note to TAs: Allow use of the notation $\langle A[i], B[j] \rangle$, $A[i] \cdot B[j]$, $\sum_{k=1}^d A[i, k] \cdot B[j, k]$ etc. to denote the dot product c of rows i and j of A and B respectively. Similarly, allow the use of the notation $\max_{1 \leq i \leq n, 1 \leq j \leq m} T[i, j]$ to denote c^* directly. Also, be careful, as the final answer (the max expression) may be calculated as a running maximum.

- (Partial marks for the orange solution) 2 points for initializing the DP table with the correct base case. 1 point **for each** for loop (n , m , and r). 1 point for calculating the current $\langle A[i], B[j] \rangle$. 2 points for filling the DP table with the correct recurrence relation. 2 points for calculating/returning the correct final answer.
- (No marks) 0 points otherwise.

(e) (5 points) Argue the runtime of your algorithm.

Filling the base cases in the table takes $O(n + m)$ time (although we *overkill* by filling the entire table with $-\infty$ in $O(nm)$ time). For each (i, j) , the calculation of the dot product between $A[i]$ and $B[j]$ takes $O(d)$ time (even if you use the summation notation in the pseudocode). There are a total of $n \times m$ (i, j) pairs. The calculation of max is an $O(1)$ operation. Thus, the overall time complexity of the algorithm is $O(nmd)$.

The exact same argument holds for the green solution. We just fill the table in the reverse order.

Alternatively, initializing the 3-dimensional table takes $O(n \cdot m \cdot \min(n, m))$ time, and we fill the base cases in $O(n + m)$ time. For each r , we move over all nm (i, j) pairs. For each (i, j) we calculate the dot product of the current rows in $O(d)$ time and fill the table. Finally, we take $O(n \cdot m \cdot \min(n, m))$ time to calculate the max over r . Then, the overall time complexity simply becomes $O(n \cdot m \cdot \min(n, m) \cdot d)$.

Alternatively, if you pre-compute the row-wise dot products in $O(nmd)$ time and store them in a table $C[i, j]$, you get $O(1)$ access to every $\langle A[i], B[j] \rangle$ inside the loops. So, the factor of $O(d)$ work is not done inside all three loops. This brings down the overall time complexity to $O(nmd + n \cdot m \cdot \min(n, m))$. For the purposes of this problem, both are acceptable.

Grading:

- (Full marks) 5 points for any correct runtime justification.
- (Partial marks for blue and green solutions) 1 point for correct time complexity of filling the base cases. 2 points for recognizing that calculating $\langle A[i], B[j] \rangle$ takes $O(d)$ time. 1 point for counting $n \times m$ (i, j) pairs. 1 point for the correct final answer.
- (Partial marks for the orange solution) 1 point for correct time complexity of filling the base cases. 1 point for recognizing that calculating $\langle A[i], B[j] \rangle$ takes $O(d)$ time. 1 point for recognizing that the loop on r runs for $\min(n, m)$ iterations in the worst case. 1 point for counting the triplets. 1 point for the correct final answer.
- (No marks) 0 points otherwise.

2. (20 points) Given an integer n and non-negative numbers $p_1, p_2, \dots, p_n \in [0, 1]$, you want to determine the probability of obtaining strictly more heads than tails when n biased coins are tossed independently at random, where p_i is the probability that the i^{th} coin comes up heads. Give an $O(n^2)$ algorithm for this task. Assume you can multiply and add two numbers in $[0, 1]$ in $O(1)$ time.

Pre-Requisites:

- For a coin, if q is the probability of obtaining heads, the probability of obtaining tails is $1 - q$.
- For independent events $Q_1, Q_2, \dots, Q_m$,

$$\mathbb{P} \left[\bigcap_{i=1}^m Q_i \right] = \prod_{i=1}^m \mathbb{P}[Q_i] \quad (12)$$

- For disjoint events $Q_1, Q_2, \dots, Q_m$,

$$\mathbb{P} \left[\bigcup_{i=1}^m Q_i \right] = \sum_{i=1}^m \mathbb{P}[Q_i] \quad (13)$$

- (a) (2 points) Give a precise description of the sub-problems you want to solve in the dynamic program.

Note: Here, we require ‘what is the sub-problem,’ and not how it was obtained or how it will be solved.

Define $T[i, j]$ as the probability of obtaining exactly i heads from the first j coin tosses, where $0 \leq i \leq n$ and $1 \leq j \leq n$.

Grading:

- (Full marks) 2 points for the correct definition. **Note to TAs:** Be careful with what i and j represent in the student’s solution.
- (Partial marks) Deduct 0.5 points for every missing condition (on i and/or j).
- (No marks) 0 points otherwise.

- (b) (2 points) Give a recurrence relation (with base case) which relates each sub-problem to “smaller” sub-problems with a one-line description of what it represents.

Note: Here, we just require the recurrence, not the algorithm or why the recurrence is correct.

We can write the following recurrence relation

$$T[i, j] = p_j \cdot T[i - 1, j - 1] + (1 - p_j) \cdot T[i, j - 1] \quad (\text{using (12) and (13)}) \quad (14)$$

which reads as heads from the j^{th} coin and $i - 1$ heads from the previous $j - 1$ coins or tails from the j^{th} coin and i heads from the previous $j - 1$ coins. The base cases of this recurrence are

$$T[i, j] = \begin{cases} \mathbb{P}[\text{No heads in } j \text{ coins}] = \prod_{k=1}^j (1 - p_k) & \text{if } i = 0 \quad (\text{using (12)}) \\ \mathbb{P}[\text{First coin flip is heads}] = p_1 & \text{if } i = 1, j = 1 \\ \mathbb{P}[\text{more heads than \# of coin flips}] = 0 & \text{if } i > j \end{cases} \quad (15)$$

Note to TAs: Be careful - i uses 0-based indexing and j uses 1-based indexing. Alternative solutions are also possible. For example, we can use 0-based indexing for both i and j , and say $\mathbb{P}[0 \text{ heads in } 0 \text{ coin flips}] = T[0, 0] = 1$. In this case, the table **must only be filled for** the cases where $i \leq j$.

Grading:

- 1 point for the correct recurrence relation. 0 points otherwise. No partial marking.

- 1 point for the correct base cases. If j uses 1-based indexing, 0.5 points if at most two cases are missing. 0 points otherwise.

(c) (1 point) Identify the sub-problem(s) that solve(s) the original problem.

Based on the definition of the sub-problem, there are multiple sub-problems that have more heads than tails for all $j = n$ coin tosses. Thus, the final answer is given by

$$\sum_{\frac{n}{2} < i \leq n} T[i, n] = \sum_{i=\lfloor \frac{n}{2} \rfloor + 1}^n T[i, n] \quad (\text{using (13)}) \quad (16)$$

Grading:

- (Full marks) 1 point for specifying the correct summation/equation. It is okay to be hand-wavy and not take *floor*, or mentioning only one of the two equations.
- (No marks) 0 points otherwise.
- No partial marking.

(d) (10 points) Give the pseudocode for implementing your dynamic program **iteratively**. A C++/Java-like program with heavy syntax, variable declarations, etc. **will not be accepted**.

Note: You do not need to prove the correctness of your algorithm in any form.

Algorithm 4 shows the iterative pseudocode for the problem.

Algorithm 4 Iterative dynamic program to find the probability of obtaining more heads than tails in n biased coin tosses

```

1: procedure PROBABILITY-OF-MORE-HEADS( $p_1, p_2, \dots, p_n$ )
2:    $T[0 : n][1 : n] \leftarrow 0$  ▷  $(n + 1) \times n$  DP table
3:   for  $j \leftarrow 1$  to  $n$  do
4:      $T[0, j] \leftarrow 1$ 
5:     for  $k \leftarrow 1$  to  $j$  do
6:        $T[0, j] \leftarrow T[0, j] \cdot (1 - p_k)$ 
7:     end for
8:   end for
9:    $T[1, 1] \leftarrow p_1$ 
10:  for  $j \leftarrow 1$  to  $n$  do
11:    for  $i \leftarrow j + 1$  to  $n$  do
12:       $T[i, j] \leftarrow 0$  ▷ Already done in initialization btw
13:    end for
14:  end for
15:  for  $j \leftarrow 2$  to  $n$  do
16:    for  $i \leftarrow 1$  to  $j$  do
17:       $T[i, j] \leftarrow p_j \cdot T[i - 1, j - 1] + (1 - p_j) \cdot T[i, j - 1]$ 
18:    end for
19:  end for
20:   $p^* \leftarrow 0$ 
21:  for  $i \leftarrow \lfloor \frac{n}{2} \rfloor + 1$  to  $n$  do
22:     $p^* \leftarrow p^* + T[i, n]$ 
23:  end for
24:  return  $p^*$ 
25: end procedure

```

Using shorthand notation (summations, products, and $\forall$ instead of for loops) is acceptable. This makes writing pseudocode easier, but be very careful while analyzing the time complexity.

Grading:

- (Full marks) 10 points for any fully correct **iterative** DP pseudocode.
- (Partial marks) 2 points for filling the first base case. 1 point for filling the second base case. 2 points for filling the third base case (be it through initialization). 3 points for filling the DP table with the correct recurrence relation. 2 points for calculating and/or returning the correct final answer.

Note to TAs: Allow use of the notation such as

$$T[i, j] \leftarrow 0 \quad \forall i > j \quad T[0, j] \leftarrow \prod_{k=1}^j (1 - p_k) \quad \forall 1 \leq j \leq n \quad p^* \leftarrow \sum_{\frac{n}{2} < i \leq n} T[i, n]$$

in the pseudocode. The 2 points for the base cases include 1 point for correct conditions/loops and 1 point for filling in the correct formula. Similarly, the 3 points for filling the recurrence include 1 point for each correct condition (on i and j) and 1 point for filling in the correct formula.

- (No marks) 0 points otherwise.

(e) (5 points) Argue the runtime of your algorithm.

The dynamic programming table is initialized in $O(n^2)$ time. The following time is spent to fill in the base cases

- $O(n^2)$ time for the first base case: while j goes from 1 to n , k goes from 1 to j , which means for $j = n$, k goes till n .
- $O(1)$ time for the second base case: we fill only one entry.
- $O(n^2)$ time for the third base case: the loops look the same as *bubble sort* - for $i = 1$, j goes from 2 to n (a distance of $O(n)$), and there are n possible values for i .

After filling in the base cases, there are approximately $\frac{n \cdot (n+1)}{2}$ (i, j) pairs remaining, each of which is filled in $O(1)$ time. Alternatively, the two for loops for filling the recurrence clearly take $O(n^2)$ iterations (since for $j = n$, i goes from 1 to n), each of which perform $O(1)$ work. Thus, this step takes $O(n^2)$ time.

Therefore, the overall time complexity of the algorithm is $O(n^2)$, which was desired.

Grading:

- (Full marks) 5 points for any correct runtime justification.
- (Partial marks) 1 point each for justifying the time taken to fill each base case. 1 point for recognizing that the recurrence loops take $O(n^2)$ iterations. 1 point for the correct final answer.
- (Partial marks) Alternatively, if j uses 1-based indexing (there is only 1 base case), 1 point for recognizing that the base case can be filled in $O(1)$ time (or during initialization). 3 points for recognizing the $O(n^2)$ time for the recurrence loops. 1 point for the correct final answer.
- (No marks) 0 points otherwise.